UNIVERSITY OF BUEA

Faculty Of Engineering and Technology

CEF440

Internet Programming and Mobile Programming

Group 16

TASK 4: SYSTEM MODELLING AND DESIGN

Submitted to:

Dr. Nkemeni Valery

2025/2026

GROUP MEMBERS

NKENG SAMA MOKOM	FE23A118
ENOW JOHN EYONG	FE23A046
CHIANGOIESEH CHRIS RAWLINGS	FE23A029
KEUKANG CARMEN ALICIA	FE23A074
NKENGAFAC WILLYHERMAN FONKEM	FE23A119

Table of Content

I. INTRODUCTION

System modeling and design is the process of creating abstract representations of a system and defining its architecture. It acts as a technical blueprint, translating user requirements into a structured model that dictates how hardware, software, and data will interact to meet performance and scalability goals.

1. System Modeling

System modeling uses graphical notations (often Unified Modeling Language or UML) to visually document a system from various perspectives.

- **External Perspective:** Models the system's environment and context.
- **Interaction Perspective:** Shows how the system interacts with users or how internal components communicate (e.g., Sequence Diagrams).
- **Structural Perspective:** Depicts the static organization, such as objects or data structures (e.g., Class Diagrams).
- **Behavioral Perspective:** Illustrates dynamic responses to events and state changes.

2. System Design

System design translates the models into a concrete architecture. It breaks down the system into manageable components and focuses on meeting specific engineering and business requirements.

- **High-Level Design (HLD):** Focuses on the overall system architecture, defining major modules, databases, and how servers or services communicate.
- **Low-Level Design (LLD):** Focuses on the detailed logic, specific algorithms, and exact data structures within individual components.
- **Key Pillars:** Emphasizes non-functional requirements like scalability, reliability, availability, and performance.

I. CONTEXT DIAGRAM

A Context Diagram is a high-level representation of a system that shows how the system interacts with external entities. It provides an overview of data flow between the system and outside components without showing internal processing details.

The context diagram presented for the QoE Monitoring App illustrates the interaction

between the application and various external entities such as the Android OS, Mobile Subscriber, Network Operator, Maps SDK, and Remote Backend Server.

1.1.Context Diagram Overview

The QoE Monitoring App acts as the central system that collects, processes, and exchanges network quality information with different entities.

1.2.External Entities and Their Roles

1.2.1. Mobile Subscriber

The Mobile Subscriber represents the end user of the application. The user interacts with the system by providing ratings and consent and viewing dashboards and alerts.

1.2.2. Android OS

The Android Operating System supplies important device information to the application, including network statistics, GPS location data, and connectivity information.

1.2.3. Network Operator

The Network Operator receives anonymous QoE reports from the application. These reports help operators analyze network quality, detect weak coverage areas, and improve service performance.

1.2.4. Maps SDK (Google/OSM)

The Maps SDK provides map tiles and geographical services that allow the application to display user locations and visualize network performance on maps.

1.2.5. Remote Backend Server

The Remote Backend Server stores synchronized data and supports cloud-based record management and analytics.

1.3.Data Flow Explanation

The following data flows are represented in the context diagram:

- Mobile Subscriber → QoE Monitoring App : Ratings & Consent.
- QoE Monitoring App → Mobile Subscriber : Dashboard & Alert.
- Android OS → QoE Monitoring App : Network & GPS Data.
- QoE Monitoring App → Network Operator : Anonymous QoE Data.
- Network Operator → QoE Monitoring App : Configuration Updates.
- Maps SDK → QoE Monitoring App : Map Tiles.
- QoE Monitoring App → Remote Backend Server : Sync Data.

1.4.Representation

The Context Diagram of the QoE Monitoring App provides a clear representation of how the application interacts with external systems and users. It highlights the major entities involved and the flow of information between them.

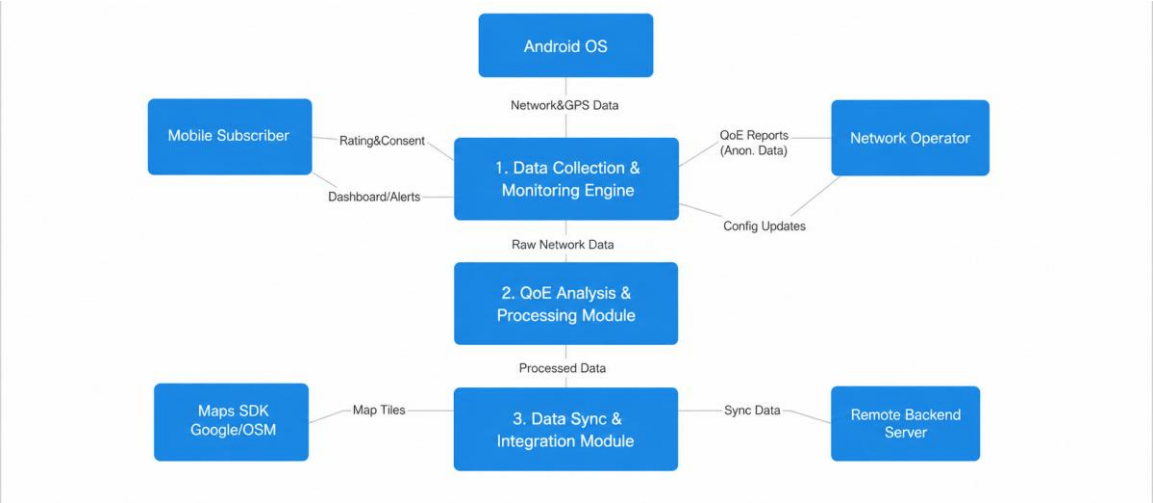


Figure 1: Context Diagram of QoE Monitoring App

1.5.Importance of the Context Diagram

The context diagram provides a clear overview of the system, identifies external entities interacting with the application, shows major data flows, and helps in understanding system boundaries.

II. DATA FLOW DIAGRAM

2.1.Level '0' Data flow Diagram

Based on the diagram below;

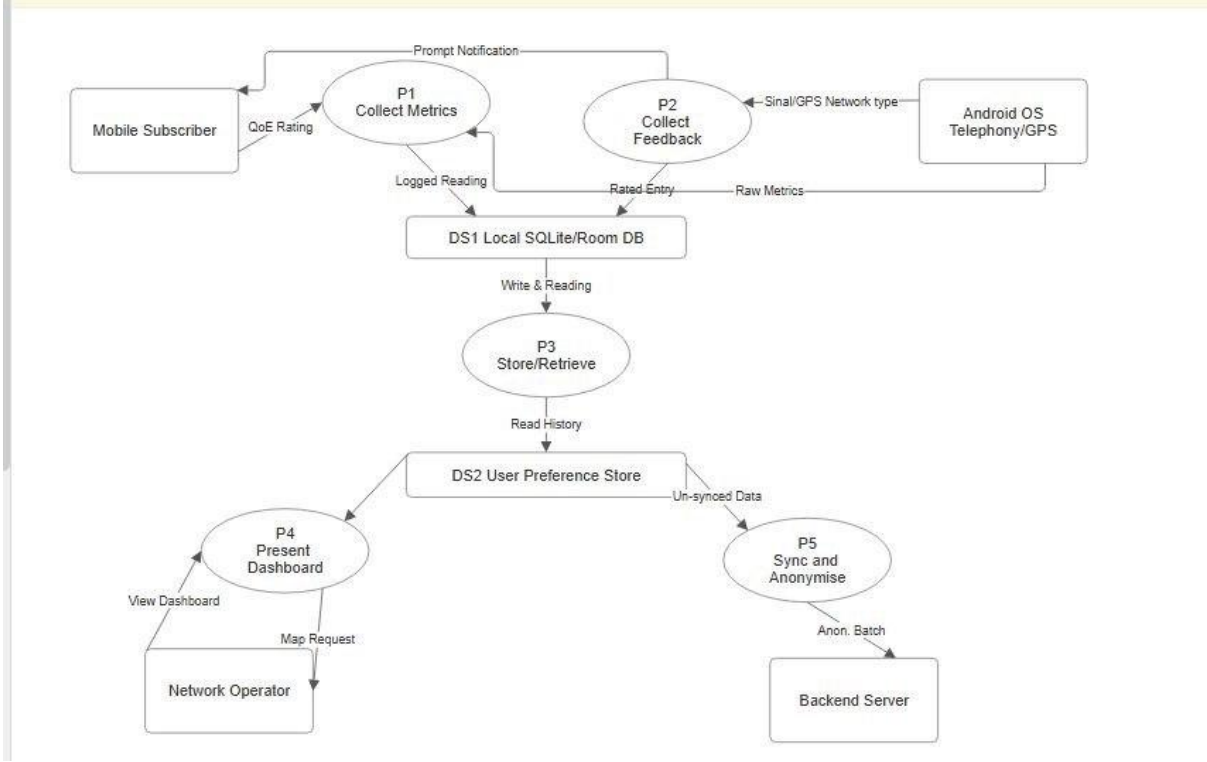
i. **Mobile Users: Users interact with the app by**
   - Running speed tests.
   - Reporting issues.
   - Viewing coverage maps.
   - Comparing operators.

ii. **Telecom operators: they received**
   - Network analytics.
   - Coverage Reports.

- Performance statistics.

iii. Government Regulators: They used the report for

- Telecom monitoring.
- Quality assurance.
- Descision making.



2.2.Level '1' Data flow Diagram

On level 1 the system is able to collect: Signal strength, Internet speed, Latency, GPS location and User feedback.

Input

For the input data collected by the systems are from:

- User device data.
- User Reports.

Output:

Raw network data stored in database

Process 2.0: Analyze Network Performance

The system collected data to:

- Identify weak signal areas
- Compares operators
- Detect poor network quality

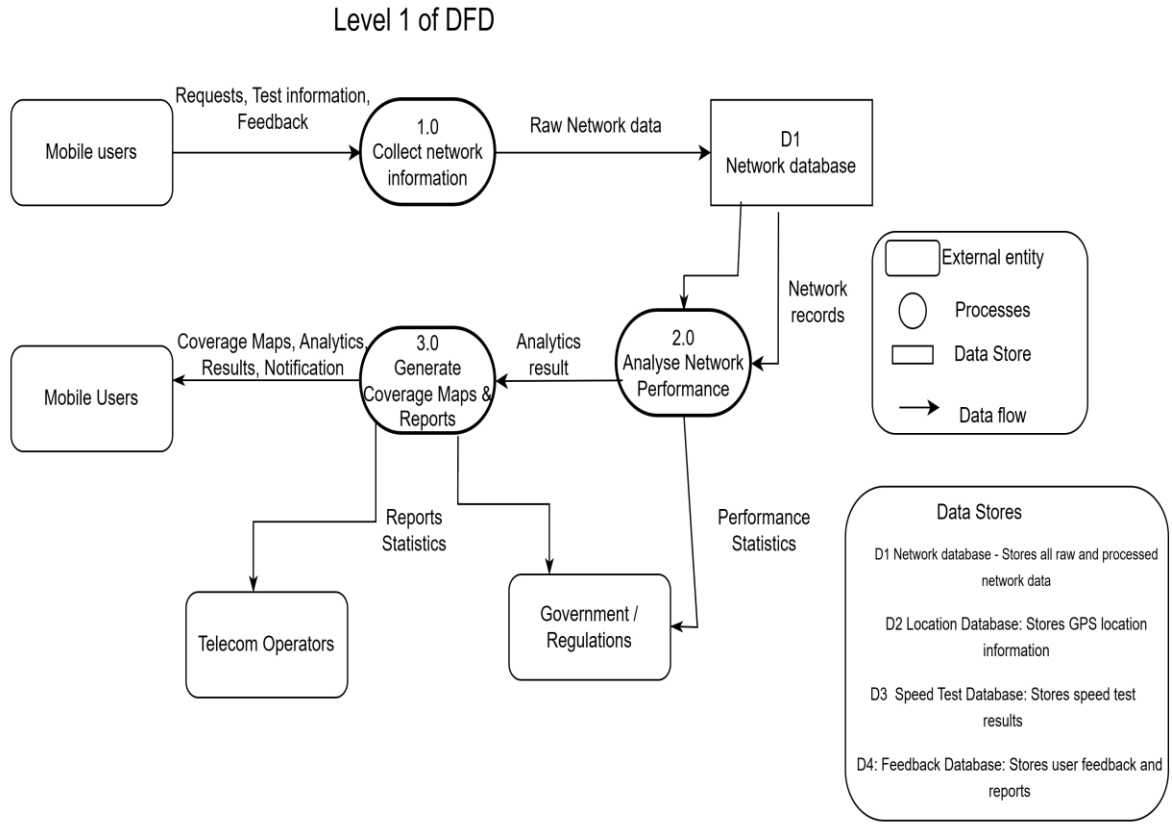
Output:

- Network analytics
- Performance statistics

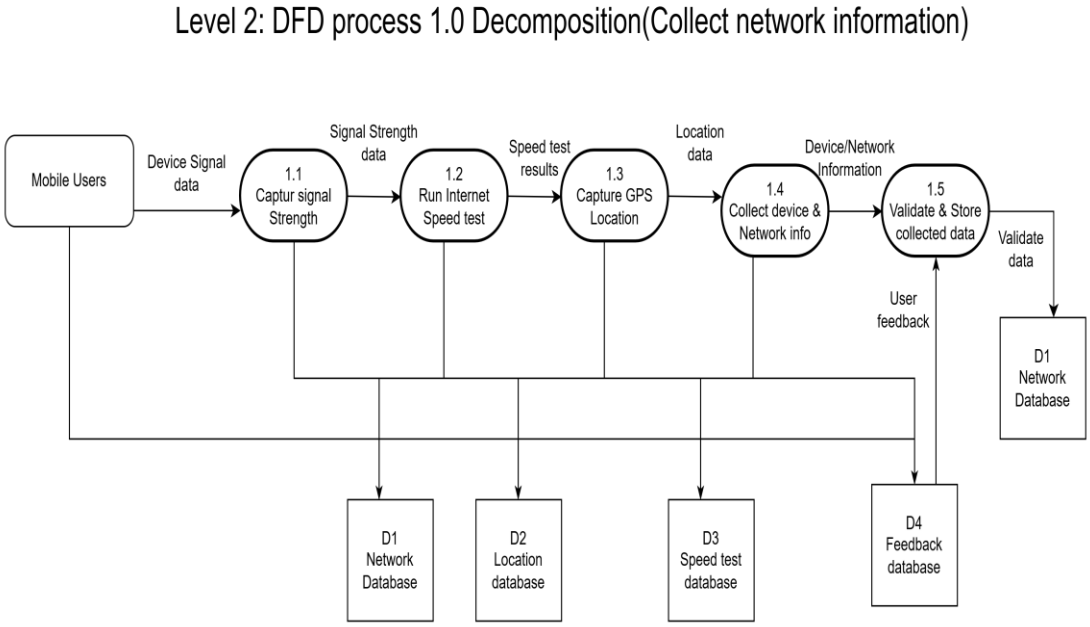
Process 3.0: Generate coverage Maps
The system uses the data and analyzes it to create:

- Coverage maps
- Signal heat maps
- Operator comparison visuals

Output:
Provide Visuals reports to users and operators

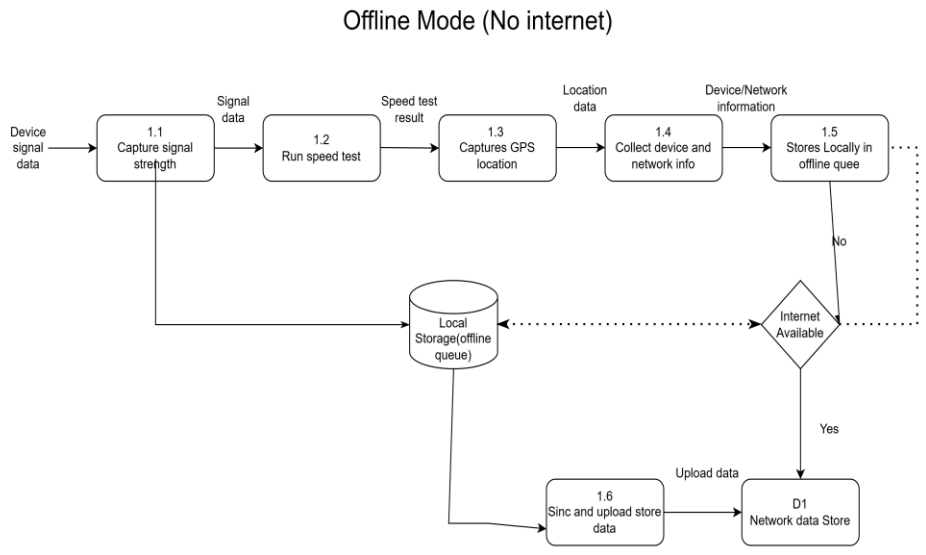


2.3.Level '2' Data flow Diagram



Data Store	Description
User database	Stores user account information
Network database	Stores signals and speed records
Coverage database	Stores generated coverage maps
Feedback Database	Stores user complains and ratings

When the users go offline from the internet, the system is still able to run from the background as seen below;



III. USE CASE DIAGRAM

3.1.Actors

In a use case diagram, actors represent the external roles that interact with a system. They define who or what is using the system, establishing clear boundaries between what is inside the software and its environment. Actors can be human users, organizations, external hardware, or other software systems.

For our system, the various actors involved with their types and descriptions are given below;

Actor	**Type**	**Description**
Mobile Subscriber	Primary	App users who generate network experience data through passive monitoring and active feedback.
System (Automated)	Internal	Background processes that collect metrics, synchronize data, and deliver notifications without user action.
Network Operator	Secondary	Telecom staff who access the admin dashboard to analyze aggregated QoE data and manage accounts.

3.2.Use Cases

A use case is a written or visual scenario describing how a user interacts with a product or system to achieve a specific goal OR A use case is a methodology used in software development and business analysis to describe how a user interacts with a system to achieve a specific goal. It outlines the sequence of actions, expected outcomes, and potential errors, ensuring the design meets real-world user needs.

For our system, the various use cases are listed below;

- **UC-01: Subscriber Installs and Onboards the App**

Actor	Mobile Subscriber
Precondition	User has downloaded the app into their mobile device
Main Flow	User opens the app for the first time. →App displays a plain-language privacy notice. →User reads and accepts the privacy notice. →App requests necessary mobile permissions (location, network state, notifications). →User grants permissions. → App presents monitoring

	mode selection (Prompted or Fully Automatic). → App begins background monitoring and displays the dashboard.
Alternative Flow	If user denies location permission, app informs them that coverage map features will be unavailable and continues with other features only.
Postcondition	Background service is running; data collection has begun.

- **UC-02: User Receives and Responds to a Feedback Prompt**

Actor	Mobile Subscriber
Precondition	User has selected Prompted Mode; prompt trigger condition has been met (e.g. once daily, or a network drop detected).
Main Flow	System sends a notification to the user's device. → User taps the notification. →App opens a simple one-screen prompt asking the user to rate their QoE → User selects fills prompt. → User submits the response. → System logs the rating, note, current metrics, and timestamp together.
Alternative Flow	If user ignores the notification, no data is logged and the next prompt is scheduled normally.
Postcondition	A combined subjective and objective QoE record is saved.

- **UC-03: User Views Dashboard and Network History**

Actor	Mobile Subscriber
Precondition	At least one metric reading has been collected and stored.
Main Flow	User opens the application. →Dashboard displays current signal strength, network type, internet speed, and QoE score. →User navigates to history view. →App displays a graph of network performance over the past 7 or 30 days. → User navigates to the coverage map. → Map displays colour-coded signal quality readings at recorded locations.
Alternative Flow	If fewer than five readings have been collected, the app displays a message indicating that more data is needed for the history and map views.
Postcondition	User has viewed their personal network performance data.

- **UC-04: Manage Account and Preferences**

Actor	Mobile Subscriber
Precondition	User is authenticated and the app is installed.
Main Flow	User navigates to Settings. →User updates monitoring mode (Prompted or Fully Automatic). →User updates language preference. → User configures notification frequency. →App saves all preferences immediately. → Background service adapts to new settings without restart.
Alternative Flow	If the user disables all notifications, prompted mode is automatically suspended and the app informs the user.
Postcondition	Updated preferences are persisted and applied to the running service.

- **UC-05: Report a Network Issue Manually**

Actor	Mobile Subscriber
Precondition	User is authenticated; app is open.
Main Flow	User taps the 'Report Issue' button. →App displays a form with issue categories (no signal, dropped call, slow data, other). →User selects an issue type and optionally writes a description. → App captures current GPS location and live metrics automatically. → User submits the report. → System stores the report with timestamp, location, and attached metrics.
Alternative Flow	If no GPS is available, user is prompted to confirm their approximate location manually.
Postcondition	A structured incident report is saved and queued for server synchronisation.

- **UC-06: View Personal Coverage Map**

Actor	Mobile Subscriber
Precondition	At least five location-tagged readings have been collected.
Main Flow	User taps the 'Coverage Map' tab. →App loads the map centred on the user's last known location. → Colour-coded markers are plotted at each location where a reading was taken. →User can tap a marker to see detailed metrics for that location. →User can filter by date range or network type.

Alternative Flow	If fewer than five readings exist, the map shows an empty state with an informational message.
Postcondition	User has explored their personal coverage history on the map.

- **UC-07: Receive In-App Notifications**

Actor	Mobile Subscriber (triggered by System)
Precondition	User has granted notification permission; background service is active.
Main Flow	System detects a trigger condition (scheduled prompt, significant signal drop, or new server message). →System constructs a notification with a relevant message. →Android delivers the notification to the device. →User sees the notification in the status bar. →User taps the notification, which deep-links into the relevant app screen.
Alternative Flow	If the user has disabled system notifications, the app displays in-app banners as a fallback.
Postcondition	User has been alerted to a relevant event and directed to the appropriate screen.

- **UC-08: Log In / Authenticate**

Actor	Mobile Subscriber
Precondition	User has completed initial onboarding; app is installed.
Main Flow	User opens the app after a session expiry. →App displays the login screen. →User enters credentials (phone number or email and PIN/password). →App sends credentials to the authentication server. →Server validates credentials and returns a session token. →App stores the token securely and navigates to the dashboard.
Alternative Flow	If credentials are invalid, app displays an error message. After three failed attempts, the account is temporarily locked and the user is prompted to reset their PIN.
Postcondition	User is authenticated; a valid session token is stored on the device.

- **UC-09: Background Network Monitoring**

Actor	System (automated — no user action required)
Precondition	Background service is active; device is powered on.

Main Flow	Every 1hour, system collects signal strength, network type, latency, bandwidth, and packet loss. →System records approximate GPS location. →System attaches a timestamp to the reading. →System stores the reading in the local database. → If internet is available, system queues data for server synchronisation.
Alternative Flow	If GPS is unavailable, last known location is recorded with an approximation flag. If internet is unavailable, data is stored locally only and synchronised when connectivity is restored.
Postcondition	A new timestamped and location-tagged metric record is saved to the local database.

- **UC-10: Sync Data to Remote Server**

Actor	System (automated)
Precondition	An internet connection is available; there are unsynchronised records in the local database.
Main Flow	System detects active internet connectivity. →System retrieves all unsynchronised records from the local database. → System batches and uploads records to the remote server via HTTPS. → Server acknowledges successful receipt. → System marks records as synchronised in the local database.
Alternative Flow	If the upload fails (timeout or server error), the system retries with exponential back-off up to three times, then schedules a retry for the next connectivity window.
Postcondition	All pending records have been uploaded and acknowledged; local records are marked as synchronised.

- **UC-11: Analyze Aggregated QoE Data**

Actor	Network Operator
Precondition	Operator is logged into the admin dashboard; subscriber data has been synchronised.
Main Flow	Operator navigates to the Analytics module. → Dashboard displays key QoE indicators: average signal strength, latency, bandwidth, and subjective ratings. → Operator applies filters (date range, region,

	network type, subscriber segment). → Dashboard refreshes with filtered results and trend charts. → Operator identifies patterns or anomalies in the data.
Alternative Flow	If no data matches the selected filters, the dashboard shows an empty state with filter suggestions.
Postcondition	Operator has reviewed filtered QoE analytics and drawn operational insights.

- **UC-12: View Network Quality Heatmap**

Actor	Network Operator
Precondition	Sufficient geo-tagged readings have been synchronised from subscribers.
Main Flow	Operator opens the Heatmap view. → System aggregates all subscriber readings by geographic cell. →Map renders colour-coded intensity zones (green = good, yellow = moderate, red = poor). → Operator can zoom and pan the map. → Operator clicks a zone to see aggregated statistics for that area.
Alternative Flow	If a zone has fewer than three unique subscriber readings, it is shown with a 'low confidence' pattern overlay.
Postcondition	Operator has identified geographic areas of strong and poor network performance.

- **UC-13: Generate and Download Reports**

Actor	Network Operator
Precondition	Operator is authenticated in the admin dashboard.
Main Flow	Operator navigates to Reports. → Operator selects a report type (weekly QoE summary, issue log, coverage analysis). → Operator configures parameters (date range, region). → Operator clicks Generate. → System compiles the report and makes it available for download. → Operator downloads the report as CSV or PDF.
Alternative Flow	If data for the selected period is incomplete, the report is generated with a data gap warning.
Postcondition	A formatted report is downloaded to the operator's device.

- **UC-14: Manage Subscriber Accounts**

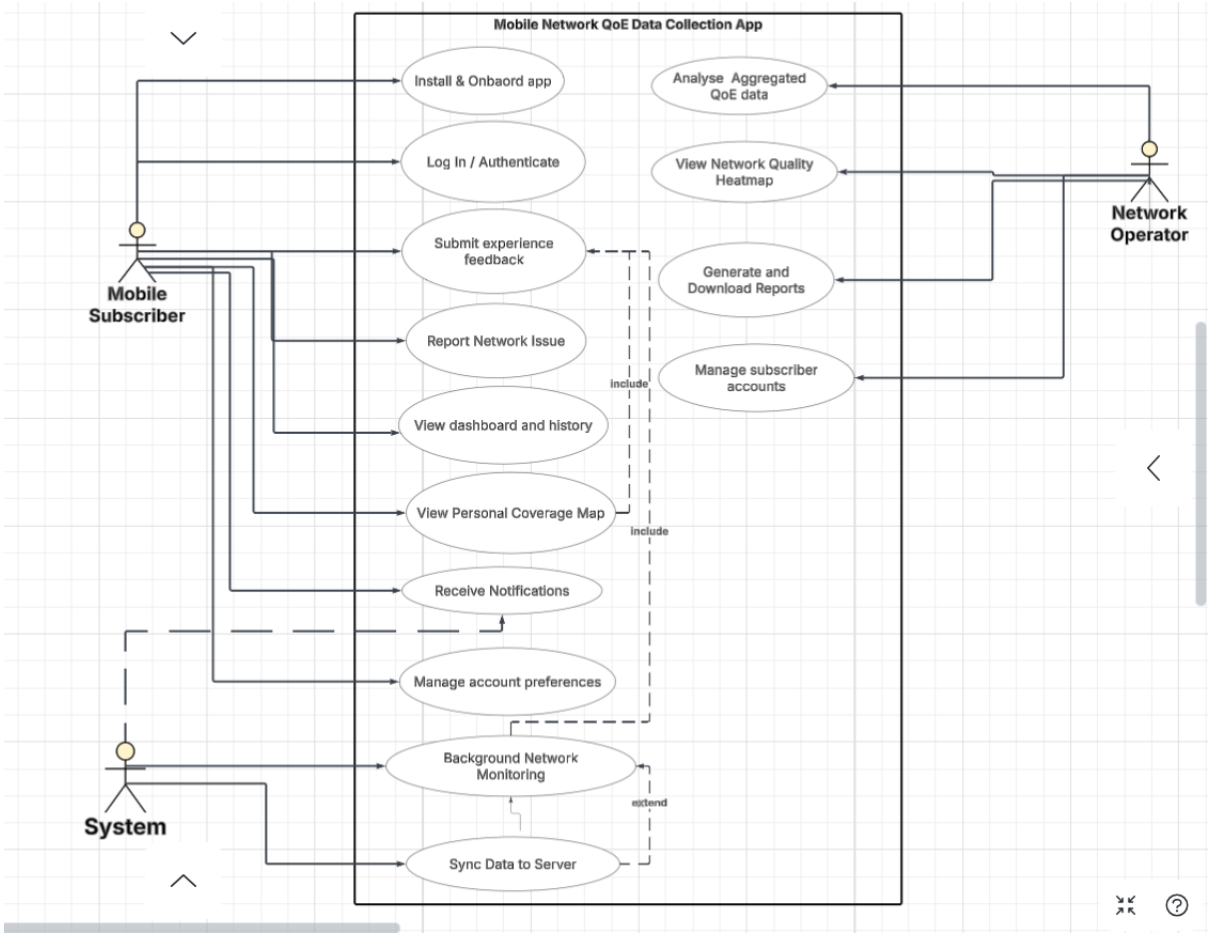
Actor	Network Operator
Precondition	Operator has administrator privileges.
Main Flow	Operator navigates to Subscriber Management. → Operator searches for a subscriber by ID, phone number, or region. → Operator views the subscriber's profile (registration date, device info, data contribution summary). → Operator can activate, deactivate, or anonymise a subscriber's account. 5. Changes are logged with the operator's ID and a timestamp.
Alternative Flow	If an operator attempts to delete a subscriber with unsynchronised data, a warning is displayed and the operation requires explicit confirmation.
Postcondition	The subscriber account reflects the changes made by the operator; all actions are audit-logged.

3.3.Use Case Diagram

A Use Case Diagram is a visual way to show how users (actors) interact with a system and what functions (use cases) the system provides. It helps understand the system's behavior from the user's perspective. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.

- Represents actors (users or external systems) and use cases (system functionalities).
- Shows the relationships between users and system features.
- Helps define system scope, requirements, and interactions clearly.

From all the above use cases, the use case diagram of our system is given below:



3.4. Why are Use Cases Important?

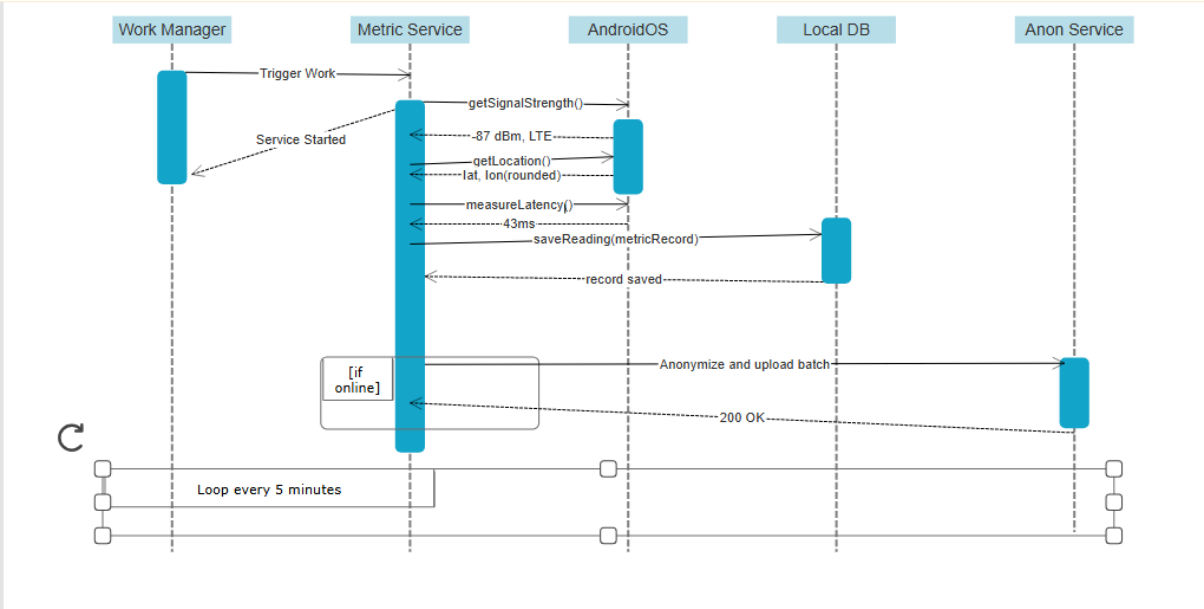
- **Requirement Gathering:** They translate abstract client ideas into concrete, actionable scenarios.
- **Development Focus:** They help software teams build features that align exactly with what the user needs.
- **Quality Assurance:** QA testers use them to map out tests and catch potential system errors early.

IV. SEQUENCE DIAGRAM

The Sequence Diagram zooms into the time dimension of a single use case specifically UC-09 (Background Network Monitoring).

- Time flows downward.

- Vertical dashed lines are lifelines representing the five components involved: WorkManager (Android's job scheduler), MetricService (the background service), Android OS (the telephony/GPS API), LocalDB (the SQLite/Room database), and AnonService (the anonymisation module).
- Horizontal arrows are messages, solid arrows are calls, dashed arrows are returns.
- Thin activation bars on lifelines show when a component is actively executing.
- The [if online] fragment and the loop note at the bottom capture the conditional and repeating nature of the cycle.
- This diagram directly implements FR-01 through FR-10 and FR-21 through FR-23 from our SRS.



V. CLASS DIAGRAM

The class diagram for the Mobile Network QoE Monitoring Application defines the static structure of the system at the code level. It identifies the key classes that make up the application, the attributes and methods each class holds, and the relationships that exist between them. Each class in this diagram represents a distinct responsibility within the system, and together they form a coherent and well-structured architecture that supports continuous background monitoring, user feedback collection, local data storage, anomaly detection, and server synchronization.

The diagram was designed following object-oriented principles, ensuring that each class has a single, clearly defined purpose, that dependencies between classes are minimized where

possible, and that the overall structure is maintainable and extensible for future development.

5.1.Class Descriptions

5.1.1. Network Reading

The class diagram for the Mobile Network QoE Monitoring Application defines the static structure of the system at the code level. It identifies the key classes that make up the application, the attributes and methods each class holds, and the relationships that exist between them. Each class in this diagram represents a distinct responsibility within the system, and together they form a coherent and well-structured architecture that supports continuous background monitoring, user feedback collection, local data storage, anomaly detection, and server synchronisation.

The diagram was designed following object-oriented principles, ensuring that each class has a single, clearly defined purpose, that dependencies between classes are minimised where possible, and that the overall structure is maintainable and extensible for future development.

5.1.2. MonitoringService

MonitoringService is the central engine of the application. It is the Android background service that runs continuously on the device, coordinating all background activity. Its attributes track whether the service is currently running, the interval between collections, the monitoring mode selected by the user, and the time of the last collection. Its methods allow it to start, stop, and restart itself, trigger metric collection, attach timestamps to readings, and schedule the next collection cycle.

MonitoringService is the most connected class in the diagram. It owns and coordinates the NetworkMetricCollector, LocationManager, AnomalyDetector, and NotificationManager through composition relationships, meaning all four of these components exist solely in the context of the MonitoringService and cannot function independently of it. It also creates NetworkReading objects, writes to the LocalDatabase, and reads from UserSettings to determine how it should behave.

5.1.3. NetworkMetricCollector

NetworkMetricCollector is responsible for one specific job that is reading network performance data directly from the Android device hardware. It uses Android's TelephonyManager and ConnectivityManager APIs to retrieve signal strength, network type, operator name, and SIM slot information, and it performs active measurements of latency, bandwidth, and packet loss. Its attributes hold references to these Android system managers

and track which SIM slot is currently active.

This class exists separately from MonitoringService because the task of reading hardware metrics is technically complex and involves multiple Android APIs. Separating this concern into its own class keeps MonitoringService clean and focused on coordination rather than low-level hardware interaction. NetworkMetricCollector is owned by MonitoringService through a composition relationship, it has no purpose or existence outside of the monitoring context.

5.1.4. **LocationManager**

LocationManager handles all GPS-related operations for the application. Its attributes track the last known latitude and longitude, whether GPS is currently available, and the accuracy radius used for anonymisation. Its methods retrieve the current location, anonymise the coordinates by rounding them to an accuracy of approximately 500 metres, return the last known location when GPS is unavailable, and check whether GPS services are currently active on the device.

LocationManager exists as a separate class because location handling involves its own set of Android permissions, availability checks, and anonymisation logic that would clutter the MonitoringService if embedded directly within it. Like NetworkMetricCollector, it is owned by MonitoringService through a composition relationship and exists solely to serve the monitoring process.

5.1.5. **AnomalyDetector**

AnomalyDetector is the intelligent component of the background monitoring system. It receives each NetworkReading as it is created and compares its metric values against predefined threshold values for signal strength, latency, and packet loss. Its attributes store these threshold values and track how many consecutive poor readings have been observed. Its methods analyse incoming readings, determine whether a network drop has occurred, trigger alerts through the NotificationManager, and reset the drop counter when quality recovers.

AnomalyDetector exists as a separate class because anomaly detection is a distinct logical concern that involves its own state, specifically the consecutive drop counter, which is necessary to distinguish a genuine sustained outage from a momentary fluctuation. Without this separation, MonitoringService would need to manage threshold logic and drop state internally, making it unnecessarily complex.

5.2.Relationship Descriptions

5.2.1. Aggregation between MonitoringService and NetworkReading

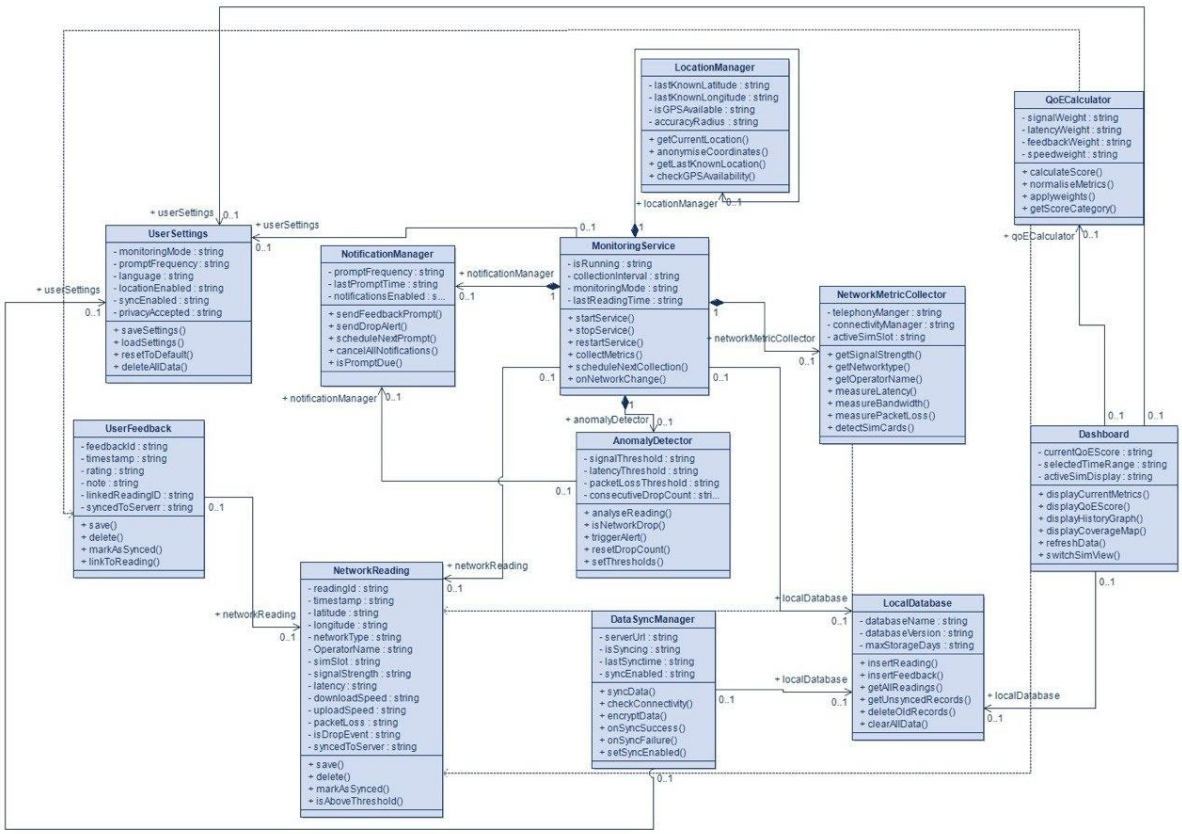
MonitoringService has an aggregation relationship with NetworkReading, with a multiplicity of one MonitoringService to many NetworkReadings. This relationship was chosen because while MonitoringService creates NetworkReading objects, those objects survive independently in the LocalDatabase long after the service has moved on to the next collection cycle. A NetworkReading does not depend on the MonitoringService for its continued existence, once created and stored, it belongs to the database. This independence distinguishes aggregation from composition and makes it the correct choice here.

5.2.2. Association between UserFeedback and NetworkReading

UserFeedback has an association relationship with NetworkReading, with a multiplicity of zero-or-one UserFeedback to one NetworkReading. This relationship exists because every UserFeedback record is linked to the specific NetworkReading that was active at the time the user submitted their rating. This linkage is fundamental to the research value of the application since it allows analysts to compare what the network was technically doing at the exact moment the user expressed satisfaction or dissatisfaction. The zero-or-one multiplicity on the UserFeedback side reflects the reality that not every NetworkReading will have a corresponding user rating, only those where the user responded to a prompt.

5.2.3. Dependency between NetworkMetricCollector and NetworkReading

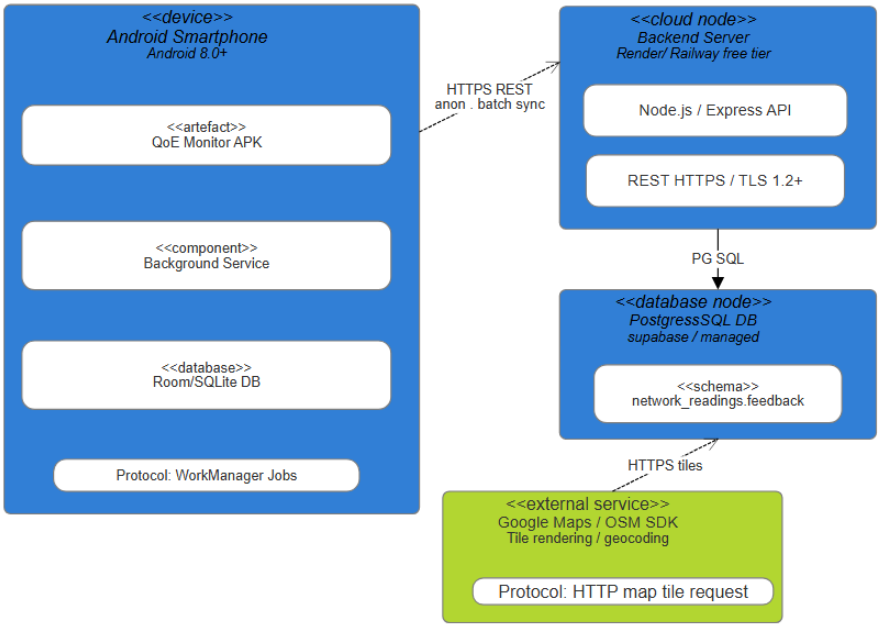
NetworkMetricCollector has a dependency relationship with NetworkReading with a multiplicity of one to many. This relationship exists because NetworkMetricCollector creates NetworkReading objects as the output of its measurement process. It does not store a permanent reference to these objects — it constructs one, populates it with the metrics it has read, and passes it back to the MonitoringService. The temporary, one-directional nature of this interaction makes dependency the correct relationship rather than association.



VI. DEPLOYMENT DIAGRAM

The Deployment Diagram answers the question "where does everything physically run?"

- It uses three stereotyped node types: «device» for physical hardware (the Android smartphone), «cloud node» for the hosted backend (Render/Railway), and «database node» for the managed PostgreSQL instance (Supabase).
- Inside each node, «artefact» and «component» boxes show what software is deployed there.
- The dashed communication paths show the protocols crossing node boundaries: the APK communicates to the backend over HTTPS REST (TLS 1.2+, satisfying NFR-13), and separately makes HTTPS tile requests to the Maps SDK.
- The Node.js API connects to PostgreSQL over the native PostgreSQL wire protocol.
- The local Room/SQLite database lives entirely within the device node, making offline operation (FR-22) physically clear no network path is needed to read/write local data.



CONCLUSION

The system modeling and design phase of the "Design and Implementation of a Mobile App for Collection of User Experience Data from Mobile Network Subscribers" establishes a robust, scalable, and secure blueprint for deployment. By translating high-level requirements into detailed system architectures—including precise Component, Deployment, and multi-level Data Flow Diagrams (DFDs)—the structural and behavioral dynamics of the platform have been rigorously defined. This modular approach ensures that the data pipeline, from real-time mobile subscriber inputs to backend analytics, remains highly performant and fault-tolerant.

A critical success factor addressed in this design is user engagement and data integrity. Given the inherent stakeholder and subscriber reluctance typically associated with digital surveys, the system architecture explicitly incorporates streamlined, low-friction user interfaces and optimized data-cleaning workflows. By structuring data flows to handle anomalies and segment responses effectively, the platform guarantees that the collected Quality of Experience (QoE) metrics are both accurate and actionable for mobile network operators. Ultimately, this design bridges the gap between raw telecommunication data and user-centric insights. The architectural frameworks established in this report provide a reliable foundation for the upcoming implementation phase, ensuring the final mobile application serves as an efficient, secure, and highly scalable solution for real-time user experience assessment.

REFERENCES

R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY: McGraw-Hill Education, 2020.

I. Sommerville, *Software Engineering*, 10th ed. Boston, MA: Pearson, 2016.

E. Yourdon, *Modern Structured Analysis*. Englewood Cliffs, NJ: Prentice-Hall, 1989.

M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA: Addison-Wesley, 2004.

L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA: Addison-Wesley, 2021.

Object Management Group (OMG), "Unified Modeling Language (OMG UML) Specification," Version 2.5.1, Dec. 2017.

B. Han, V. Gopalakrishnan, and S. Sen, "Mobile data crowdsourcing: A review of systems, applications, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 1, pp. 577–603, First Quarter 2016.

R. K. Ganti, F. Ye, and H. Lei, "Mobile crowdsensing: Current state and future challenges," *IEEE Communications Magazine*, vol. 49, no. 11, pp. 32–39, Nov. 2011.

J. Schulz and M. Seufert, "A reference architecture for distributed mobile crowdsensing platforms," in *Proc. Int. Conf. Soft. Eng. and Advanced Apps*, 2019, pp. 112–119.